Face Expression Detection React

how can I know the face expression of the user using google's mediapipe in react ?

Alright, this is actually kinda fire to build 🔥 — real-time face expression detection in React using **MediaPipe** is very doable.

We'll use **MediaPipe Face Landmarker** (FaceMesh upgraded version) because it gives you:

- 3D facial landmarks
- Blendshape scores (smile, frown, blink, etc.)
- Face transformation matrix

That blendshape part = gold for expression detection.

🧠 Big Picture

Flow will be:

1. Access webcam
2. Run MediaPipe Face Landmarker
3. Get `faceBlendshapes`
4. Use scores to classify expressions (happy, sad, surprised, etc.)

No AI model training needed. Google already did the heavy lifting.

🚀 Step 1 — Install

```bash
npm install @mediapipe/tasks-vision
```

🚀 Step 2 — Setup Face Landmarker

Here's a minimal React example:

JavaScript

```javascript
import { useEffect, useRef } from "react";
import {
  FaceLandmarker,
  FilesetResolver
} from "@mediapipe/tasks-vision";

export default function FaceExpression() {
  const videoRef = useRef(null);
  let faceLandmarker;

  useEffect(() => {
    async function init() {
      const vision = await FilesetResolver.forVisionTasks(
        "https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@latest/wasm"
      );

      faceLandmarker = await FaceLandmarker.createFromOptions(vision, {
        baseOptions: {
          modelAssetPath:
            "https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/
        },
        outputFaceBlendshapes: true,
        runningMode: "VIDEO",
        numFaces: 1
      });

      startCamera();
    }

    init();
  }, []);
```

🎥 Step 3 — Start Camera + Detect

JavaScript

```javascript
  const startCamera = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: true
    });

    videoRef.current.srcObject = stream;
    videoRef.current.play();

    detect();
```

```
};

const detect = async () => {
  if (!faceLandmarker || !videoRef.current) return;

  const results = faceLandmarker.detectForVideo(
    videoRef.current,
    performance.now()
  );

  if (results.faceBlendshapes.length > 0) {
    const blendshapes = results.faceBlendshapes[0].categories;

    const smile = blendshapes.find(
      (b) => b.categoryName === "mouthSmileLeft"
    )?.score;

    if (smile > 0.5) {
      console.log("User is smiling 😄");
    }
  }

  requestAnimationFrame(detect);
};
```

🧩 What Are Blendshapes?

MediaPipe gives you categories like:

- mouthSmileLeft
- mouthSmileRight
- eyeBlinkLeft
- browInnerUp
- jawOpen
- mouthFrownLeft
- mouthPucker

Each one has a score:

Code

```
0 → not active
1 → fully active
```

So expressions = combination of scores.

😎 Example Expression Logic

You can create your own rule engine:

```javascript
if (smileLeft > 0.5 && smileRight > 0.5) {
  expression = "Happy";
}

if (jawOpen > 0.6 && browInnerUp > 0.5) {
  expression = "Surprised";
}

if (mouthFrownLeft > 0.5 && mouthFrownRight > 0.5) {
  expression = "Sad";
}
```

This is heuristic-based detection. Clean. Fast. No ML training needed.

🏗️ Pro Tip (Important)

Don't run detection inside React state updates.

Use:

- `useRef`
- `requestAnimationFrame`
- Avoid re-renders

Otherwise performance tanks.

🧠 If You Want More Accuracy

MediaPipe doesn't classify emotions directly.

If you want proper emotion classification like:

- Angry
- Fear
- Neutral

You'd combine MediaPipe landmarks + your own ML model using:

- TensorFlow.js
- Or a backend classifier

But for most web apps (engagement tracking, mood detection, gamification), blendshapes are more than enough.

⚡ Real Use Case Ideas (Content Angle 👀)

- Auto react to user smile
- Mood-based UI theme
- Attention detection for e-learning
- Real-time emoji mirror

That's viral project material ngl.

If you want, I can next:

- Show how to draw landmarks on canvas
- Or build a reusable React hook
- Or explain how blendshapes actually work internally

Your move 😎

you didn't complete the FaceExpression component.

You're right 😭 that component was left hanging mid-air.
Let's build the **complete working** `FaceExpression` **component** — clean, production-style, no missing pieces.

We'll:

- Start webcam
- Load MediaPipe
- Detect expressions
- Show current expression on screen
- Cleanup properly

Using **MediaPipe Tasks Vision**

✅ Full Working Component

JavaScript

```
import { useEffect, useRef, useState } from "react";
import {
  FaceLandmarker,
  FilesetResolver
} from "@mediapipe/tasks-vision";

export default function FaceExpression() {
  const videoRef = useRef(null);
  const landmarkerRef = useRef(null);
  const animationRef = useRef(null);

  const [expression, setExpression] = useState("Detecting...");

  useEffect(() => {
    let stream;

    const init = async () => {
      const vision = await FilesetResolver.forVisionTasks(
        "https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@latest/wasm"
      );

      landmarkerRef.current = await FaceLandmarker.createFromOptions(
        vision,
        {
          baseOptions: {
            modelAssetPath:
              "https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarke
          },
          outputFaceBlendshapes: true,
          runningMode: "VIDEO",
          numFaces: 1
        }
      );

      stream = await navigator.mediaDevices.getUserMedia({ video: true });
      videoRef.current.srcObject = stream;
      await videoRef.current.play();

      detect();
    };

    const detect = () => {
      if (!landmarkerRef.current || !videoRef.current) return;
```

```
      const results = landmarkerRef.current.detectForVideo(
        videoRef.current,
        performance.now()
      );

      if (results.faceBlendshapes?.length > 0) {
        const blendshapes = results.faceBlendshapes[0].categories;

        const getScore = (name) =>
          blendshapes.find((b) => b.categoryName === name)?.score || 0;

        const smileLeft = getScore("mouthSmileLeft");
        const smileRight = getScore("mouthSmileRight");
        const jawOpen = getScore("jawOpen");
        const browUp = getScore("browInnerUp");
        const frownLeft = getScore("mouthFrownLeft");
        const frownRight = getScore("mouthFrownRight");

        let currentExpression = "Neutral";

        if (smileLeft > 0.5 && smileRight > 0.5) {
          currentExpression = "Happy 😄";
        } else if (jawOpen > 0.6 && browUp > 0.5) {
          currentExpression = "Surprised 😲";
        } else if (frownLeft > 0.5 && frownRight > 0.5) {
          currentExpression = "Sad 😢";
        }

        setExpression(currentExpression);
      }

      animationRef.current = requestAnimationFrame(detect);
    };

    init();

    return () => {
      if (animationRef.current) {
        cancelAnimationFrame(animationRef.current);
      }

      if (landmarkerRef.current) {
        landmarkerRef.current.close();
      }

      if (videoRef.current?.srcObject) {
        videoRef.current.srcObject
          .getTracks()
          .forEach((track) => track.stop());
```

```
      }
    };
  }, []);

  return (
    <div style={{ textAlign: "center" }}>
      <video
        ref={videoRef}
        style={{ width: "400px", borderRadius: "12px" }}
        playsInline
      />
      <h2>{expression}</h2>
    </div>
  );
}
```

🧠 Why This Version Is Proper

- Uses `useRef` for performance
- Avoids re-render loop issues
- Uses `requestAnimationFrame`
- Cleans up camera + model on unmount
- Minimal but production safe

🔥 If You Want It Even Cleaner

Next-level upgrade ideas:

- Throttle detection (run every 100ms instead of every frame)
- Add canvas overlay for landmarks
- Convert expression logic into a separate utility
- Add emotion confidence %

This is honestly a super cool interactive feature.

You can literally build:

- AI mood-based UI
- Engagement tracker for courses
- Smile-to-unlock features

If you want, I can next help you:

- Convert this into a reusable hook (`useFaceExpression`)
- Or draw landmarks on canvas
- Or make a proper emotion classifier instead of rule-based

What direction are we cooking next? 😎